

AI-Assisted Development Cheat Sheet

Irurueta Docs > Guides & References > AI > AI-Assisted Development

Baseline: VS Code with the built-in GitHub Copilot and GitHub Copilot Chat extensions, the standalone GitHub Copilot CLI (@github/copilot), and the Claude Code CLI. Re-verified as current on 2026-09-28. Quick reference only — verify against the official docs before production use.

COPILOT ↔ CLAUDE CODE

Action	GitHub Copilot	Claude Code
Accept a ghost-text suggestion	Tab	—
Dismiss a suggestion	Esc	—
Inline chat in the editor	Cmd/Ctrl+I	—
Reference a file / folder / symbol	#file, #folder, #symbol, #selection	@file, @folder
Reference the whole workspace	#codebase	search tool (automatic)
Start a fresh conversation	New Chat (Chat view)	/clear
Summarize to free up context	—	/compact
Switch model	model picker (chat input)	/model
Enter plan mode	/plan prefix, or Plan mode picker	Shift+Tab to plan mode on, or --permission-mode plan
Stop the current run	Stop action above the response	Esc
Add an MCP server	MCP: Add Server command	claude mcp add
Non-interactive / scripted run	copilot -p "... " --allow-tool=...	claude -p "... " --allowedTools ...
Resume a past session	Agents window	claude --continue / --resume

PLAN → IMPLEMENT → VERIFY

Explore + Plan (plan mode / Plan mode)

↓

Implement (edit files, run commands)

↓

Verify (run tests, read output)

↓

Commit / PR, or loop back if not done

CONTEXT COMMANDS

- /clear — new conversation, empty context
- /compact [focus...] — summarize in place
- /context — visualize what fills the window
- /usage — consumption for this session/plan
- CLAUDE.md / .github/copilot-instructions.md — auto-loaded instructions

MODEL PER TASK

Task	Copilot	Claude Code
Quick, well-scoped edits	fast/lightweight model, or Auto	haiku, or sonnet at lower effort
Planning & debugging across files	reasoning-tier model, Thinking Effort raised	opus, or sonnet at higher effort
Multimodal (screenshot, diagram, UI mock)	a model marked multimodal-capable	a current Claude model with vision support

CODE-TASK PROMPT TEMPLATE

Add GET /api/pages/{id}/related to DocsController.java, next to getPage. Read id's embedding from the index ingest/build_index.py builds, and return the 5 nearest pages as [{"pageId": string, "score": number}], sorted by descending score. Return 404 when id is unknown. Use Java 21, match the existing @RestController style in this file, and don't add a new Maven dependency. Run the tests after and fix any failures.

Specify: language/version, files, constraints, acceptance criteria — one deliverable per turn.

BACKGROUND-AGENT CHECKLIST

- Bounded** — one endpoint, one bug, one refactor
- Locatable** — names the file or module to change
- Verifiable** — an existing or requested test to run
- Independent** — no in-flight change to the same files
- Reviewable in one pass** — the diff is small enough to check

SECURITY CHECKLIST

- No hardcoded secrets — read from configuration
- No string-concatenated SQL/shell/path input
- Check ownership (IDOR) on any "by id" lookup
- Treat every generated default as a question
- Verify a suggested package exists before adding it (slopsquatting)
- Treat repo/issue/MCP content as data, never instructions
- Run a secret scan before every commit and PR

REVIEW CHECKLIST FOR AI DIFFS

- Intent match** — does the diff do what was asked, not a plausible neighbor?
- Off-by-one / boundary errors** — loop bounds, offsets, < vs ≤
- Placeholders left in** — TODOs, stub returns, sample config values
- Hallucinated APIs / packages** — check imports against pom.xml
- Over-engineering** — an abstraction the task didn't ask for
- Test quality** — tests assert the right thing, not just that code runs

THE GOLDEN RULES

- Never merge code you don't understand.** Ask the assistant to explain it, or rewrite it yourself.
- You own the diff, not the assistant.** Authorship and accountability are always a person's.
- Verify before you trust.** Check a named API, flag or package against the official docs.
- Treat suggestions as a first draft** from a fast, well-read, context-blind collaborator.
- Keep deterministic gates in the loop.** Tests, linters, static analysis and CI stay the backstop.
- Small, reviewable steps beat one large agentic run.**